Guia de Estudo: Implementação do PAPT em Julia com JuMP e CPLEX

Sumário

1 Objetivo deste guia 2

2 Visão geral do código da orientadora 2

3 Bibliotecas e parâmetros gerais 2

4 Leitura e preparação dos dados 3

5 Função objetivo 3

6 Variáveis de decisão 4

7 Configuração do solver 4

8 Tradução das restrições para JuMP 5

9 Exportação, resolução e leitura da solução 5

10 Observações sobre o arquivo recebido 6

11 Plano de desenvolvimento recomendado 6

12 Primeiro teste e valores de referência 6

13 Checklist de estudo 6

1 Objetivo deste guia

Este material explica como partir da formulação matemática do Problema de Alocação de Professores a Turmas (PAPT) e construir uma implementação em Julia com JuMP e CPLEX. O código enviado pela orientadora serve como referência de organização, mas aparenta ser um trecho de um projeto maior, com conjuntos e parâmetros definidos fora do arquivo apresentado. A proposta é aproveitar a estrutura geral sem copiar diretamente as variáveis específicas daquele problema.

2 Visão geral do código da orientadora

O programa pode ser compreendido em etapas:

1. importar bibliotecas;
2. ler e organizar os dados da instância;
3. definir a função objetivo;
4. criar o modelo e as variáveis;
5. configurar o solver;
6. adicionar as restrições;
7. resolver o modelo;
8. extrair e registrar a solução.

3 Bibliotecas e parâmetros gerais

O código começa com:

```
using JuMP, CPLEX, DataStructures, DelimitedFiles,
      Statistics, MathOptInterface, Combinatorics
```

Biblioteca	**Função**
`JuMP`	Escrever modelos matemáticos de otimização em Julia.
`CPLEX`	Interface para utilizar o solver IBM CPLEX.
`DelimitedFiles`	Ler arquivos de texto delimitados.
`MathOptInterface`	Interface padronizada entre JuMP e solvers.
`DataStructures`, `Statistics`, `Combinatorics`	Estruturas e operações auxiliares usadas em outras partes do projeto.

O código também define:

```
const ZERO = 0.0001
const NTHREADS = 8
```

No trecho enviado, `ZERO` é usado como tolerância de gap relativo do CPLEX e `NTHREADS` define o número de threads. O gap é um critério de parada para o problema inteiro misto; não representa uma tolerância geral aplicada a todas as variáveis e restrições. Para o primeiro protótipo do PAPT, comece com uma configuração simples e só depois experimente parâmetros avançados do solver. Assim, erros de formulação podem ser separados dos efeitos de configuração.

4 Leitura e preparação dos dados

O exemplo usa:

```
instance = ARGS[1]
const ninstance = parse(Int, instance)

filepath = "/home/pcandida/punim0040/instances/coordenadasSP.txt"
populationDP = readdlm(string(filepath, instance, "DP.txt"))
```

`ARGS[1]` recebe o primeiro argumento passado ao executar o script. Isso permite selecionar uma instância sem alterar o código. O caminho absoluto, porém, depende do computador da orientadora. No PAPT, organize:

- P: professores;
- T: turmas;
- K: disciplinas;
- S: slots semanais fixos;
- R: combinações de dia e turno;
- h_j: carga horária semanal da turma j;
- w_{ij}: preferência do professor i pela turma j;
- e_{ij}: elegibilidade do professor i para a turma j.

Separe dados institucionais observados dos parâmetros gerados para os experimentos. O professor registrado na oferta original é uma referência da alocação atual, não necessariamente a lista de todos os professores elegíveis para a turma.

5 Função objetivo

No código da orientadora, a função objetivo é encapsulada em:

```
function setObjective(m)
    # ...
    @objective(m, Min, ...)
end
```

A macro `@objective` define o sentido da otimização e a expressão a otimizar. O exemplo minimiza penalidades e custos. Para o PAPT, a versão não lexicográfica e sem normalização discutida anteriormente utiliza:

$$\max Z = 0{,}60F(x) + 0{,}20\underline{s} - 0{,}10C(z) - 0{,}10\overline{g}. \tag{1}$$

Os critérios são:

$$F(x) = \sum_{i \in P} \sum_{j \in T} w_{ij} x_{ij}, \tag{2}$$

$$C(z) = \sum_{i \in P} \sum_{r \in R} z_{ir}. \tag{3}$$

Em JuMP:

```
@expression(model, F,
    sum(w[(i,j)] * x[i,j] for i in P for j in T)
)
@expression(model, C,
    sum(z[i,r] for i in P for r in R)
)
@objective(model, Max,
    0.60 * F +
    0.20 * s -
    0.10 * C -
    0.10 * g_max
)
```

Definir os critérios como expressões separadas facilita registrar seus valores individualmente. Como não há normalização, os pesos precisam ser avaliados, pois os critérios podem ter escalas diferentes.

6 Variáveis de decisão

No código recebido, as variáveis são criadas com `@variable`. A palavra `Bin` indica uma variável binária e `>= 0` define um limite inferior. Para o PAPT:

```
@variable(model, x[i in P, j in T], Bin)
@variable(model, z[i in P, r in R], Bin)
@variable(model, g[i in P, k in K], Bin)
@variable(model, s >= 0)
@variable(model, g_max >= 0, Int)
```

Variável	**Significado**
x_{ij}	Vale 1 se o professor i for alocado à turma j; caso contrário, 0.
z_{ir}	Vale 1 se o professor i tiver aula em pelo menos uma turma no turno semanal r.
g_{ik}	Vale 1 se o professor i lecionar ao menos uma turma da disciplina k.
$\underline{s}$	Piso da satisfação dos professores incluídos no critério de equidade.
$\overline{g}$	Maior número de disciplinas distintas atribuído a um professor.

Os nomes `s` e `g_max` são nomes de implementação e representam $\underline{s}$ e $\overline{g}$.

7 Configuração do solver

O código recebido configura o CPLEX com:

```
set_optimizer_attribute(m, "CPX_PARAM_THREADS", NTHREADS)
set_optimizer_attribute(m, "CPX_PARAM_EPGAP", ZERO)
set_optimizer_attribute(m, "CPX_PARAM_TILIM", 3600)
```

Esses parâmetros controlam threads, gap relativo e limite de tempo. As opções de heurísticas e cortes são configurações do solver, não restrições matemáticas. Para o primeiro protótipo, mantenha configurações essenciais e introduza ajustes avançados somente depois de validar o modelo.

8 Tradução das restrições para JuMP

A macro `@constraint` permite declarar restrições. A cobertura exige:

$$\sum_{i \in P} x_{ij} = 1, \qquad \forall j \in T. \tag{4}$$

Em JuMP:

```
@constraint(model, cobertura[j in T],
    sum(x[i,j] for i in P) == 1
)
```

A expressão `j in T` cria uma restrição para cada turma.

Restrição matemática	**Tradução conceitual**
Cobertura	Exatamente um professor por turma.
Elegibilidade	$x_{ij} \leq e_{ij}$.
Carga mínima	A soma das horas atribuídas ao professor é pelo menos $h_i^{\min}$.
Carga máxima	A soma das horas atribuídas ao professor não ultrapassa $h_i^{\max}$.
Piso de satisfação	$\underline{s} \leq \sum_j w_{ij} x_{ij}$ para cada professor considerado.
Ligação turma–disciplina	$x_{ij} \leq g_{i,\mathrm{Disc}(j)}$.
Teto de disciplinas	$\overline{g} \geq \sum_k g_{ik}$.
Conflito por slot	Cada professor assume no máximo uma turma em cada slot fixo.
Compactação por turno	Se uma turma alocada ocorre no turno r, então $z_{ir} = 1$.

As ligações reversas de g_{ik} e z_{ir} também devem ser implementadas. Elas garantem que as variáveis auxiliares representem a alocação real, em vez de permanecerem artificialmente maiores que a ocupação efetiva.

9 Exportação, resolução e leitura da solução

O código da orientadora exporta o modelo:

```
write_to_file(m, "./model/ps" * instance * ".lp")
```

O arquivo `.lp` permite inspecionar a formulação construída. A resolução é iniciada com:

```
optimize!(model)
```

Após a resolução, verifique o status do solver antes de acessar valores das variáveis. Para benchmarks, registre:

- status da otimização;
- valor da função objetivo;
- alocação de cada turma;
- carga semanal de cada professor;
- satisfação individual;
- número de turnos ocupados;
- máximo de disciplinas por professor;
- tempo de execução e gap, quando disponíveis.

10 Observações sobre o arquivo recebido

O trecho enviado aparenta depender de código que não está incluído. Conjuntos e parâmetros como `K`, `T`, `P`, `priority` e `demand` são utilizados sem definição no fragmento. Além disso, alguns acessos às variáveis `V` e `Q` aparentam usar quantidades de índices diferentes das declarações apresentadas. Esses pontos devem ser conferidos no projeto completo. Assim, o arquivo é mais útil como referência de organização do que como programa autônomo.

11 Plano de desenvolvimento recomendado

Etapa A. Instância pequena fixa. Codificar diretamente em Julia os três professores e as quatro turmas da instância reduzida.

Etapa B. Validação matemática. Conferir cobertura, elegibilidade, carga horária, conflitos, satisfação e variáveis auxiliares.

Etapa C. Entrada JSON. Depois que o protótipo funcionar, carregar o JSON normalizado produzido pelo Python.

Etapa D. Benchmarks. Automatizar leitura das instâncias, construção do modelo, resolução e registro dos resultados.

Essa sequência reduz fontes possíveis de erro. Com dados escritos diretamente no código, divergências tendem a estar na tradução matemática ou nas restrições. A leitura do JSON pode ser adicionada depois, como etapa independente.

12 Primeiro teste e valores de referência

O exemplo inicial utiliza a instância reduzida e a função objetivo ponderada sem normalização. Os valores de referência para a solução indicada no estudo anterior são:

Indicador	**Valor de referência**
Preferência total F	17
Piso de satisfação $\underline{s}$	3
Turnos ocupados C	8
Máximo de disciplinas $\overline{g}$	1
Função objetivo Z	9,9

Esses valores servem como referência para conferir a implementação; não constituem confirmação de que o arquivo foi executado nesta etapa. A execução depende de Julia, JuMP, CPLEX e de uma instalação/licença compatível do solver.

13 Checklist de estudo

- ☐ Sei explicar a função de JuMP e CPLEX.
- ☐ Sei distinguir conjuntos, parâmetros e variáveis de decisão.
- ☐ Consigo traduzir a restrição de cobertura para `@constraint`.
- ☐ Consigo explicar por que x_{ij}, z_{ir} e g_{ik} são diferentes.
- ☐ Sei por que o modelo deve ser validado com uma instância pequena.
- ☐ Sei quais indicadores registrar em cada benchmark.